

University of Applied Sciences Hamburg	Digital Systems	 Hochschule für Angewandte Wissenschaften Hamburg <i>Hamburg University of Applied Sciences</i>
Group/Table No: 1/7	Lab 2 Room 8.01	Efe Kardes Matriculation No: 2757072
Date: 20/11/2025		Emir Avni Sahin Matriculation No: 2756541
Professor: Peter Schulz		Sajjad Rahman Matriculation No: 2613734
CODEC Datapath		

Report History

Report 1		Date		Remarks
	received			
	checked			
	result	o.k.		
		n.g.		

Prof.

Contents

1	Introduction	2
2	Summary of Preparatory Tasks	2
2.1	Prep Task 1: Understanding the Design	2
2.2	Prep Task 2: Signal Domains and Timing	2
2.3	Prep Task 3: Simulation	3
2.4	Prep Task 4: Logic Design	3
3	Laboratory Setup	3
4	Presentation and Discussion of Results	4
4.1	Task 1.1: Signal Loop-back Verification	4
4.1.1	Measurement Configuration	4
4.1.2	Latency Measurement Methodology	4
4.2	Task 1.2: Datapath Logic Verification	6
4.2.1	Exchange Test	6
4.2.2	Negation Tests	7
5	Summary	9
A	VHDL Source Code	11
A.1	Physical Constraints (XDC)	11
A.2	Design Entity: codec_interface.vhd	12

1 Introduction

In this laboratory session, our main objective was to design a system that connects an FPGA (Field Programmable Gate Array) to an external audio chip, known as a CODEC. The CODEC digitizes incoming analog audio, the FPGA processes the 16-bit samples, and sends them back to the CODEC for digital-to-analog conversion.

A key challenge in this experiment is that the FPGA is responsible for generating all timing signals required by the CODEC. The CODEC sends data bit by bit (Serial), while the FPGA processes data 16 bits at a time (Parallel). We needed to build a bridge between these two domains.

Our work involved three main steps:

- **Clock Generation:** Creating the specific timing signals required to keep the audio chip synchronized with the FPGA.
- **Data Conversion:** Designing modules to convert serial data into parallel numbers (SIPO) and parallel numbers back into serial data (PISO).
- **Signal Processing:** Implementing logic to manipulate the audio, such as swapping the stereo channels or inverting the signal phase.

2 Summary of Preparatory Tasks

2.1 Prep Task 1: Understanding the Design

Before the lab, we analyzed the provided VHDL code to understand its structure. The design operates using a "loop-back" principle. Data comes in from the ADC (Analog-to-Digital Converter), goes through our FPGA logic, and is immediately sent back to the DAC (Digital-to-Analog Converter).

The system relies on a central Clock Controller. This module takes the fast 100 MHz system clock and divides it down to create the slower clocks needed for audio sampling. Internally, the FPGA does not use these clocks directly. Instead, the divided clocks generate clock-enable pulses, allowing all logic (SIPO, PISO, datapath) to remain synchronized in the single 100 MHz clock domain.

2.2 Prep Task 2: Signal Domains and Timing

We identified that the system requires three distinct clock signals to function correctly:

- **LRC (Left/Right Clock):** Running at approx. 48 kHz, this signal tells the system whether the current data belongs to the left channel or the right channel.
- **Bit Clock:** This fast signal coordinates the movement of individual data bits.
- **System Clock:** This is the master reference for the CODEC chip itself.

A crucial detail we noted is how these signals are handled inside the FPGA. While 'LRC', 'Bit Clock', and 'System Clock' act as actual clocks for the external CODEC chip, they are **not** used to clock the internal logic of the FPGA.

2.3 Prep Task 3: Simulation

We used a testbench to simulate the design on the computer. Since simulation cannot process real analog audio, the testbench emulates the CODEC's digital behavior by generating a fixed 32-bit serial input stream synchronized to the BIT and LRC clocks. This allows us to verify that the SIPO (Serial-In-Parallel-Out) and PISO (Parallel-In-Serial-Out) modules correctly shift and regroup audio samples in the FPGA before testing the design on real hardware.

2.4 Prep Task 4: Logic Design

We designed two combinational logics to process the audio:

- **Exchange Logic:** A control signal from the IOM board selects whether the left and right channel samples are passed normally or swapped using a simple multiplexer.
- **Negation Logic:** Audio channel can be inverted by applying a unary minus to the signed 16-bit sample. Because the audio data represents positive and negative amplitudes, we used the signed datatype in VHDL to ensure the mathematical negation operates correctly.

The complete VHDL source code derived from this preparation task is included in **Appendix A**.

3 Laboratory Setup

The hardware setup for this experiment consisted of the following components:

- **FPGA Board:** ModSys Artix-7 development board, acting as the central digital processing unit. It generated the required audio clocks (MCLK, BCLK, LRCLK), configured the PCM3006 via control signals.
- **Signal Source:** A laboratory function generator was used with two active outputs. Channel 1 provided a $1 V_{pp}$ sinusoidal test signal connected to the left input of the CODEC board. Channel 2 generated a square-wave reference signal connected to the right input.
- **Measurement:** A digital oscilloscope was used to monitor both the analog inputs to the PCM3006 and the corresponding analog outputs. The four oscilloscope channels were connected as follows:
 - **Channel 1 (Oscilloscope):** monitoring the sinusoidal input signal from Function Generator CH1.
 - **Channel 2 (Oscilloscope):** monitoring the square-wave input signal from Function Generator CH2.
 - **Channel 3 (Oscilloscope):** observing the left-channel analog output.
 - **Channel 4 (Oscilloscope):** observing the right-channel analog output.

The oscilloscope allowed observation of amplitude consistency, output waveform quality, and potential phase shift or latency between input and output.

4 Presentation and Discussion of Results

4.1 Task 1.1: Signal Loop-back Verification

In the first task, we programmed the FPGA with the basic loop-back design. Our goal was to verify that the audio signal could pass through the system intact and to measure the time delay.

4.1.1 Measurement Configuration

To analyze the system, we monitored four signals on the oscilloscope (see Figures 1, 2, and 3).

- **Sine Waves (Channel 1 & 3):**

The Yellow trace (Channel 1) is the input audio, and the Orange trace (Channel 3) is the output audio. We used sine waves here to check the **fidelity of the signal**.

- **Square Waves (Channel 2 & 4):**

The Green trace (Channel 2) is the digital input, and the Purple trace (Channel 4) is the digital output. We used square waves specifically to measure the **time delay**.

4.1.2 Latency Measurement Methodology

To accurately quantify the system’s total propagation delay, we utilized square waves rather than sine waves for the timing analysis. This choice was driven by the signal characteristics:

- **Ambiguity of Sine Waves:** Sinusoidal signals exhibit a gradual slope, which introduces uncertainty when determining the exact reference point for a time measurement.
- **Precision of Square Waves:** Square waves possess a steep transient response (fast rise time). This distinct edge allowed us to place the oscilloscope cursors precisely on the rising flank of the digital input (Green) and the digital output (Blue), ensuring an accurate Δt reading.

Using this method, we determined the total system latency to be:

$$\Delta t \approx 635.7 \mu s$$

As illustrated in the subsequent figures, we repeated this measurement at 100 Hz, 500 Hz, and 1 kHz. **The measured delay remained invariant across all frequencies.**

This constant latency confirms that the system operates deterministically. The total delay is the cumulative result of the fixed hardware pipeline stages—specifically the ADC’s decimation filter, the FPGA’s synchronous shift registers, and the DAC’s interpolation filter—and is independent of the input signal frequency.



Figure 1: Oscilloscope measurement at 100 Hz with the delay of $\Delta t \approx 635.7 \mu s$

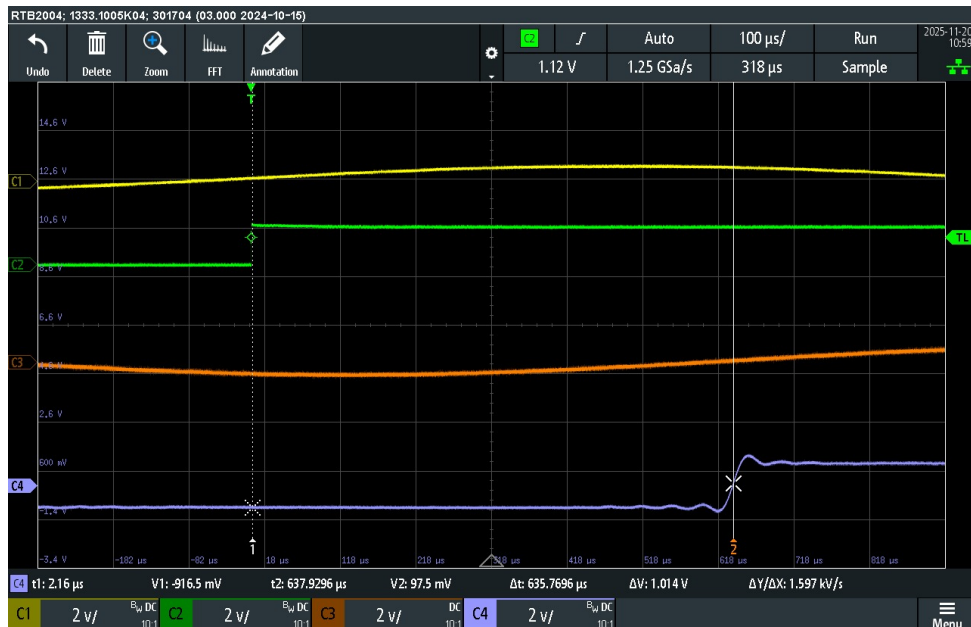


Figure 2: Oscilloscope measurement at 500 Hz with the delay of $\Delta t \approx 635.7 \mu s$

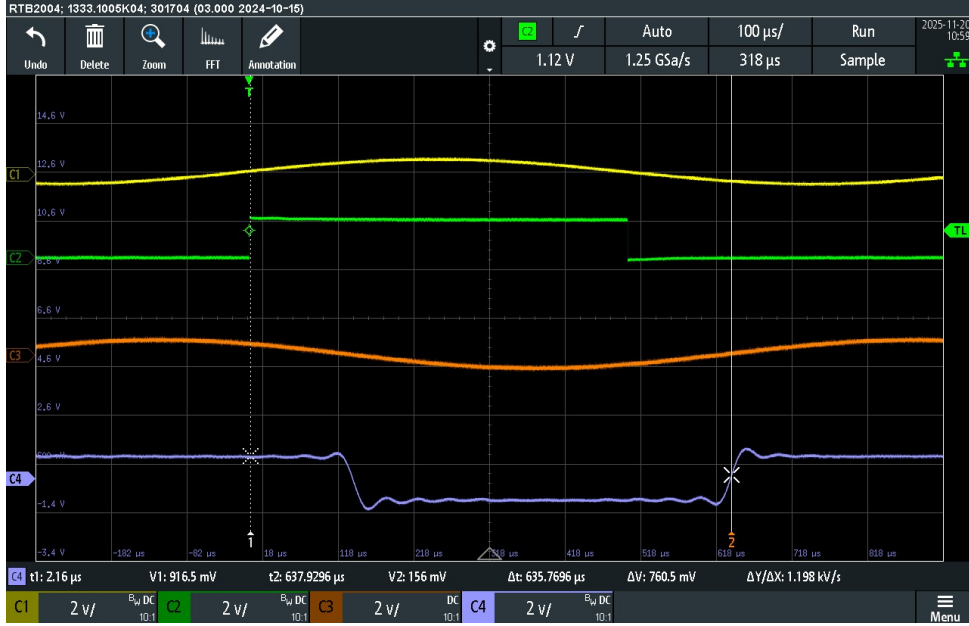


Figure 3: Oscilloscope measurement at 1kHz with the delay of $\Delta t \approx 635.7 \mu s$

4.2 Task 1.2: Datapath Logic Verification

In Task 1.2, the pure loop-back datapath from Task 1.1 was extended by two combinational logic modules in the parallel domain: a Left/Right channel exchange and a per-channel negation. Both functions are controlled via the IOM board switches and operate on the 16-bit samples between the SIPO and PISO modules.

In the following sections, the behavior of each function is verified using oscilloscope measurements.

4.2.1 Exchange Test

The exchange logic swaps the left and right channels under control of the signal `exch_lr`. In VHDL, this is implemented using two multiplexers operating on the signed representations of the SIPO outputs:

```
signal_sel_left_s  <= right_s when exch_lr = '1' else left_s;
signal_sel_right_s <= left_s  when exch_lr = '1' else right_s;
```

To verify this function, a low-frequency sine wave was applied to the **left** analog input of the audio codec, while a digital TTL signal from the function generator was applied to the **right** input. When `exch_lr = 1`, the two output channels visibly swapped, confirming correct operation of the exchange logic. The relevant measurement is shown in Figure 4.

- **Channel 1** shows the analog Left Channel input to the CODEC.
- **Channel 2** shows the analog Right Channel input to the CODEC.
- **Channel 3** displays the Left Channel analog output from the DAC.
- **Channel 4** displays the Right Channel analog output from the DAC.

When `exch_lr` is set high, the channels are swapped inside the FPGA datapath. After the rising edge of the control signal, the sine wave from Channel 1 appears on the Right Output (Channel 4), and the square wave from Channel 2 appears on the Left Output (Channel 3).



Figure 4: Oscilloscope measurement for Task 1.2: Verification of the Left/Right exchange logic. The control signal `exch_lr` (green) causes the audio signal to swap from the Left Output to the Right Output.

4.2.2 Negation Tests

The negation logic inverts the polarity of the signal. The 16-bit samples are cast to signed values and multiplied by -1 (two's complement negation).

```
out_left_s  <= -signal_sel_left_s  when neg_left  = '1' else
    signal_sel_left_s;
out_right_s <= -signal_sel_right_s when neg_right = '1' else
    signal_sel_right_s;
```

The resulting signed values are converted back to `std_logic_vector` and fed into the PISO. Two measurements were carried out to demonstrate the effect: one with a sinusoidal input and one with a rectangular input.

In this measurement, Channel 2 shows the square-wave input signal applied to the Right Channel, while Channel 4 displays the corresponding output of the codec after processing. For this test, the negation function is enabled in the FPGA. As a result, the output waveform on Channel 4 is the inverted version of the input on Channel 2: every high level becomes low, and every low level becomes high. This confirms that the implemented negation logic in Figure 5 correctly flips the signal around the zero axis.



Figure 5: Negation test with square-wave input.

In the experiment on Figure 6, Channel 1 displays the sinusoidal input applied to the Left Channel, while Channel 3 shows the corresponding output signal after processing in the FPGA. For this test, the negation function is enabled for the sinusoidal channel. Consequently, the waveform on Channel 3 is the inverted form of the input on Channel 1: every positive half-wave appears as a negative half-wave, and every negative portion is mirrored to the positive side.

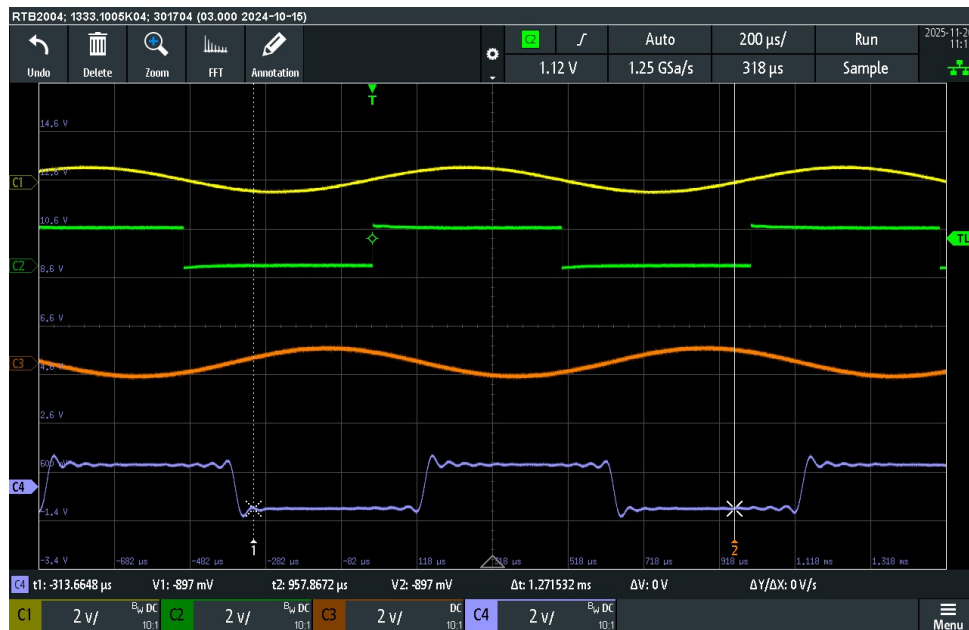


Figure 6: Negation test with sinusoidal input.

Taken together, the measurements for the exchange and negation tests demonstrate that the extended datapath of Task 1.2 behaves as intended. The FPGA can correctly swap the stereo channels and invert the polarity of each channel independently under the control of the IOM switches while preserving signal shape and timing.

5 Summary

This laboratory session focused on implementing and verifying a complete audio-processing pipeline between the Artix-7 FPGA and the PCM3006 stereo CODEC. The preparatory tasks provided the required theoretical foundation: understanding the serial audio interface, analyzing the VHDL design structure, and becoming familiar with the generated timing signals and the serial-to-parallel data conversion.

Experimental verification in Task 1.1 confirmed high signal fidelity in the loop-back path. By utilizing square waves to analyze the system's transient response, we determined a constant, frequency-independent system latency of approximately $\Delta t \approx 635.7 \mu\text{s}$. This delay is attributed to the internal digital filters of the CODEC and the buffering stages within the FPGA.

Finally, in Task 1.2, we demonstrated the FPGA's capability to manipulate audio data in the parallel domain. By applying distinct stimuli (Sine and Square waves) to the stereo inputs, we visually verified the correct operation of the channel exchange logic. Furthermore, the successful signal inversion confirmed the correct application of VHDL signed arithmetic for real-time audio processing. The system performed as intended, satisfying all timing and functional requirements.

References

- [1] L. Leutelt, *Digital Systems Lecture Slides and Lab Descriptions*, HAW Hamburg, Oct. 2025.
- [2] Burr-Brown/Texas Instruments, “PCM 3006 - 16-Bit, Single-Ended Analog Input/Output Stereo Audio Codec,” Data Sheet SBAS089A, Oct. 2004.
- [3] L. Leutelt, *DS07 – Handling Multiple Clock Domains in Digital Systems*, Hamburg University of Applied Sciences, Lecture Slides, 2025.
- [4] P. Schulz, *Digital Systems Laboratory – Audio CODEC Interface (PCM3006) Lab Description*, Department of Electrical Engineering, HAW Hamburg, 2025.
- [5] Texas Instruments / Burr-Brown, *PCM3006 – 16-Bit, Single-Ended Analog Input/Output Stereo Audio Codec*, Datasheet SBAS089A, Oct. 2004. Available: <https://www.ti.com/lit/ds/sbas089a/sbas089a.pdf>
- [6] Xilinx Inc., *7-Series FPGAs Clocking Resources User Guide*, UG472, v1.14, Mar. 2020. Available: https://docs.xilinx.com/v/u/en-US/ug472_7Series_Clocking
- [7] Xilinx Inc., *Vivado Design Suite – Synthesis User Guide*, UG901, 2023.
- [8] IEEE Standard 1164-2019, *IEEE Standard for VHDL Language Reference Manual*, Institute of Electrical and Electronics Engineers, 2019.
- [9] IEEE Standard 1076.3-2009, *VHDL Synthesis Packages (numeric_std)*, Institute of Electrical and Electronics Engineers, 2009.
- [10] Aldec Inc., *Riviera-PRO HDL Simulator – User Manual*, Aldec Documentation Release 2023.
- [11] Lab Group 1.7, *Experimental Oscilloscope Measurements*, Digital Systems Laboratory, HAW Hamburg, Oct. 20, 2025.

Statement on the Use of AI Tools

Artificial intelligence (AI) tools were used in limited and transparent ways during the preparation of this report. The following tools and assistance were employed:

- **Rewriting and Grammar Support:** ChatGPT was used to refine the readability of selected paragraphs without altering the technical content or conclusions.
- **Text Clarification and Short Explanations:** AI tools were used to obtain short clarifications, alternative phrasings, and concise summaries to support the author in expressing certain concepts more effectively. All technical explanations were written and cross-checked by the author.
- **Structural and Organizational Suggestions:** AI assistance was used to recommend improvements to the structure of the document, including section order, paragraph organization, and transitions.

- **Reference and Formatting Assistance:** AI tools were used to help create, format, and standardize bibliographic entries (BibTeX) and to ensure consistent citation style throughout the document.

A VHDL Source Code

A.1 Physical Constraints (XDC)

The following constraints define the pin mappings for the Artix-7 FPGA. This listing combines the main system settings, the audio codec interface (Connector 3), and the IOM board user I/O (Connector 4).

```

1 #####
2 # 1. MAIN SYSTEM (Clock & Reset)
3 #####
4 # Main FPGA clock (100 MHz)
5 set_property PACKAGE_PIN N5 [get_ports clk_in]
6 set_property IOSTANDARD LVCMOS33 [get_ports clk_in]
7 create_clock -period 10.000 [get_ports clk_in]
8
9 # System Reset (Active Low)
10 set_property PACKAGE_PIN P5 [get_ports reset_n]
11 set_property IOSTANDARD LVCMOS33 [get_ports reset_n]
12 set_false_path -from [get_ports reset_n]
13
14 #####
15 # 2. AUDIO CODEC INTERFACE (Connector 3)
16 #####
17 # Serial Data TO Codec
18 set_property PACKAGE_PIN K15 [get_ports {DOUT}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {DOUT}]
20
21 # Serial Data FROM Codec
22 set_property PACKAGE_PIN A11 [get_ports {DIN}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {DIN}]
24
25 # System Clock (Output to Codec)
26 set_property PACKAGE_PIN C12 [get_ports {clk_SYS}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {clk_SYS}]
28
29 # Bit Clock (Output to Codec)
30 set_property PACKAGE_PIN D14 [get_ports {clk_BIT}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {clk_BIT}]
32
33 # Left/Right Clock (Output to Codec)
34 set_property PACKAGE_PIN B14 [get_ports {clk_LRC}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {clk_LRC}]
36
37 #####
38 # 3. IOM BOARD USER I/O (Connector 4)
39 #####
40 # Global Enable Switch (IN0)
41 set_property PACKAGE_PIN R7 [get_ports {enable}]
42 set_property IOSTANDARD LVCMOS33 [get_ports {enable}]
43
44 # Heartbeat LED (OUT0)

```

```

45 set_property PACKAGE_PIN U9 [get_ports {clk_d}]
46 set_property IOSTANDARD LVCMOS33 [get_ports {clk_d}]
47
48 # --- Task 1.2 Control Switches ---
49 # Exchange Left/Right Channel (Mapped to IN1)
50 set_property PACKAGE_PIN R5 [get_ports {exch_lr}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {exch_lr}]
52
53 # Negate Left Channel (Mapped to IN2)
54 set_property PACKAGE_PIN R6 [get_ports {neg_left}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {neg_left}]
56
57 # Negate Right Channel (Mapped to IN3)
58 set_property PACKAGE_PIN N4 [get_ports {neg_right}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {neg_right}]

```

Listing 1: Combined Physical Constraints File

A.2 Design Entity: codec_interface.vhd

The following code implements the full CODEC interface, including the clock dividers, SIPO/PISO registers, and the datapath logic for channel exchange and negation (Task 1.2).

```

1
2 entity codec_interface is
3     -- Attention: generic parameters may be overwritten through
4     -- instantiation!!!
5     generic (
6         clk_in_frequency : integer := 100000000; -- 100 MHz on FPGA
7         board
8         LRC_frequency    : integer := 100000000/2048; -- LRC: left/right
9         clock
10        BIT_frequency    : integer := 100000000/64; -- Bit Clock
11        SYS_frequency    : integer := 100000000/8; -- CODEC system clock
12        d_frequency      : integer := 1 -- utility clock, e.g. "heart
13        beat"
14    );
15    port(
16        reset_n    : in std_logic;
17        enable     : in std_logic;
18        clk_in     : in STD_LOGIC; -- 100 MHz from FPGA board
19        clk_LRC    : out STD_LOGIC; -- LRC: left/right clock
20        clk_BIT    : out STD_LOGIC; -- Bit Clock
21        clk_SYS    : out STD_LOGIC; -- CODEC system clock
22        clk_d      : out STD_LOGIC; -- utility clock, connect to an LED
23        DOUT       : out STD_LOGIC; -- serial data output to CODEC
24        DIN        : in STD_LOGIC;  -- serial data input from CODEC
25
26        -- Task 1.2 Ports
27        exch_lr    : in std_logic;  -- exchange L/R channels
28        neg_left   : in std_logic;  -- negate left channel
29        neg_right  : in std_logic;  -- negate right channel
30    );
31 end codec_interface;
32
33 architecture test of codec_interface is
34     -- clock divider constant values

```

```

31  constant max_LRC : integer := clk_in_frequency / LRC_frequency;
32  constant max_BIT : integer := clk_in_frequency / BIT_frequency;
33  constant max_SYS : integer := clk_in_frequency / SYS_frequency;
34  constant max_d    : integer := clk_in_frequency / d_frequency;
35
36  constant toggle_LRC : integer := max_LRC/2; -- toggle points for 50%
duty cycle
37  constant toggle_BIT : integer := max_BIT/2;
38  constant toggle_SYS : integer := max_SYS/2;
39  constant toggle_d    : integer := max_d/2;
40
41  -- enable signals, only active for one clk_in period
42  signal clk_en_LRC_rising : std_logic;
43  signal clk_en_LRC_falling : std_logic;
44  signal clk_en_BIT_rising : std_logic;
45  signal clk_en_BIT_falling : std_logic;
46  signal clk_en_SYS_rising : std_logic;
47  signal clk_en_d          : std_logic;
48
49  -- shift register control signals
50  signal SIPO_save : std_logic;
51  signal PISO_load : std_logic;
52
53  -- SIPO registers
54  signal SI_reg      : std_logic_vector(31 downto 0); -- input shift
register
55  signal PO_reg_left : std_logic_vector(15 downto 0); -- left channel
parallel
56  signal PO_reg_right : std_logic_vector(15 downto 0); -- right
channel parallel
57
58  -- PISO register and input signals
59  signal SO_reg      : std_logic_vector(31 downto 0); -- output shift
register
60  signal PI_left     : std_logic_vector(15 downto 0); -- left channel
parallel
61  signal PI_right    : std_logic_vector(15 downto 0); -- right channel
parallel
62
63  -- Datapath Signals (Task 1.2)
64  signal left_s      : signed(15 downto 0); -- signed raw left signal
65  signal right_s     : signed(15 downto 0); -- signed raw right signal
66
67  signal out_left_s   : signed(15 downto 0); -- signed output left
signal
68  signal out_right_s  : signed(15 downto 0); -- signed output right
signal
69
70  signal signal_sel_left_s : signed(15 downto 0);
71  signal signal_sel_right_s : signed(15 downto 0);
72
73  begin
74
75  -- start of datapath operational area
76  -- "unclocked" combinational logic only !!!!!
77
78  -- 1. convert std_logic_vector -> signed
79  left_s <= signed(PO_reg_left);

```

```

80     right_s <= signed(P0_reg_right);
81
82     -- 2. exchange channels if selected
83     signal_sel_left_s  <= right_s when exch_lr = '1' else left_s;
84     signal_sel_right_s <= left_s  when exch_lr = '1' else right_s;
85
86     -- 3. negation
87     out_left_s  <= -signal_sel_left_s  when neg_left  = '1' else
signal_sel_left_s;
88     out_right_s <= -signal_sel_right_s when neg_right = '1' else
signal_sel_right_s;
89
90     -- 4. convert signed -> std_logic_vector for PISO
91     PI_left  <= std_logic_vector(out_left_s);
92     PI_right <= std_logic_vector(out_right_s);
93
94     -- end of datapath operational area
95
96     ...

```

Listing 2: VHDL Design: codec_interface.vhd